

Máster en Inteligencia Artificial
para Liderazgo Técnico y Transformación Empresarial

RNNs, LSTM y Procesamiento de Secuencias

Módulo 4 · Deep Learning

Redes recurrentes, LSTM con compuertas, GRU, BiRNN, series temporales industriales y State Space Models como sucesor eficiente.

1. Introducción · 2. Qué es · 3. Cómo funciona · 4. Ejemplos reales
5. Errores comunes · 6. Aplicación práctica · 7. Relación con módulos · 8. Resumen ejecutivo

Guía de estudio detallada · +2.000 palabras

RNNs, LSTM y Procesamiento de Secuencias

Antes de los Transformers, las redes recurrentes dominaban todos los problemas de secuencias: traducción automática, reconocimiento de voz, predicción de series temporales, generación de texto. En 2017 los Transformers los desbancaron para la mayoría de las tareas de NLP. Sin embargo, en 2025, las RNNs y los LSTMs siguen siendo la solución correcta para un conjunto importante de problemas empresariales: predicción de series temporales industriales, análisis de datos de sensores IoT, modelado de secuencias cortas con recursos computacionales limitados, y casos donde el procesamiento online (un punto de datos a la vez, sin conocer el futuro) es un requisito.

Además, en 2024 surgió una nueva familia de arquitecturas que generaliza las ideas de las RNNs: los State Space Models (SSMs), con Mamba como su implementación más notable. Mamba promete la calidad de los Transformers con la eficiencia lineal de las RNNs, y está ganando terreno especialmente en el procesamiento de secuencias muy largas donde el coste cuadrático de la atención es prohibitivo.

Redes Recurrentes: procesar secuencias con memoria

La RNN simple: estado oculto como memoria

Una RNN procesa una secuencia un elemento a la vez. En cada paso t , recibe el elemento actual x_t y el estado oculto del paso anterior h_{t-1} , y produce el nuevo estado oculto $h_t = \tanh(W_h \times h_{t-1} + W_x \times x_t + b)$. El estado oculto actúa como "memoria" que acumula información de todos los pasos anteriores. La misma función (los mismos pesos W_h y W_x) se aplica en cada paso temporal, lo que hace que el modelo sea eficiente en parámetros pero limitado en la longitud de la memoria efectiva.

El problema del gradiente desvaneciente es la limitación fundamental de las RNNs simples: al propagar el gradiente hacia atrás a través de muchos pasos temporales, el gradiente se multiplica repetidamente por los pesos W_h . Si esos pesos tienen valores <1 , el gradiente colapsa a cero y los pasos tempranos de la secuencia dejan de influir en el entrenamiento. En la práctica, las RNNs simples no pueden capturar dependencias de más de 10-20 pasos hacia atrás.

LSTM: compuertas para controlar la memoria a largo plazo

Las LSTM (Long Short-Term Memory, Hochreiter y Schmidhuber, 1997) resuelven el problema del gradiente desvaneciente con un mecanismo de compuertas que controla qué información se retiene y cuál se descarta. Una celda LSTM tiene tres compuertas: la compuerta de olvido (forget gate) decide qué información del estado de celda anterior se descarta: $f_t = \text{sigmoid}(W_f \times [h_{t-1}, x_t] + b_f)$. La compuerta de entrada (input gate) decide qué información nueva se añade al estado de celda. La compuerta de salida (output gate) decide qué parte del estado de celda se usa para el estado oculto de salida.

El estado de celda (C_t) es el "highway" de largo plazo: fluye a través del tiempo con modificaciones controladas por las compuertas. Los gradientes pueden fluir por este highway sin desvanecerse, permitiendo capturar dependencias de cientos o incluso miles de pasos. Las GRU (Gated Recurrent Units) son una versión simplificada de LSTM con dos compuertas en lugar de tres y sin estado de celda separado: menor coste computacional con rendimiento similar en muchos casos.

SECCIÓN 3 · CÓMO FUNCIONA

Arquitecturas bidireccionales y apiladas

Las RNNs unidireccionales solo "ven" el pasado en cada posición. Las Bidirectional RNNs (BiRNN) procesan la secuencia en ambas direcciones (de izquierda a derecha y de derecha a izquierda) y concatenan los estados ocultos resultantes. Esto permite que cada posición tenga contexto tanto del pasado como del futuro, lo que es muy útil para tareas de comprensión (NER, POS tagging, clasificación de texto) donde el contexto completo está disponible. No puede usarse para generación autoregresiva porque requiere toda la secuencia.

Las RNNs apiladas (stacked RNNs) añaden múltiples capas recurrentes donde el estado oculto de una capa es la entrada de la siguiente. Esto aumenta la capacidad del modelo para capturar representaciones más abstractas, análogamente a como apilar capas convolucionales en CNNs captura patrones de mayor nivel.

State Space Models: el sucesor eficiente de las RNNs

Mamba (Gu y Dao, 2024) es el representante más importante de los State Space Models (SSMs). Su innovación clave es el selective state space: en lugar de tener compuertas fijas como LSTM, las compuertas son función de la entrada, permitiendo al modelo decidir dinámicamente qué información retener o descartar. El resultado es una arquitectura que procesa secuencias en tiempo lineal ($O(n)$ en lugar de $O(n^2)$ de los Transformers) y que en benchmarks de secuencias largas supera o iguala a los Transformers con mucho menos cómputo.

SECCIÓN 4 · EJEMPLOS REALES

- Predicción de demanda en supply chain con LSTM: empresas como Amazon y Walmart usan LSTMs para predicción de demanda de SKUs individuales, capturando patrones temporales complejos (estacionalidad, efectos de promociones, dependencias entre productos) con horizontes de predicción de 1-12 semanas.
- Detección de anomalías en maquinaria industrial (Siemens): LSTMs entrenados sobre datos de sensores (temperatura, vibración, presión, corriente) de turbinas y motores para detectar desviaciones del comportamiento normal. El modelo aprende el patrón temporal normal y alerta cuando la secuencia actual se desvía significativamente.
- Reconocimiento de escritura a mano (Google): BiLSTMs para procesar secuencias de puntos (x,y,t) de gestos de escritura. Cada punto es un elemento de la secuencia, y la red aprende a mapear esas secuencias de movimientos a caracteres reconocibles.

- Análisis de trayectorias y comportamiento de usuario (Spotify): LSTMs sobre secuencias de reproducciones para modelar las preferencias musicales del usuario como función del contexto temporal. La canción que quieres escuchar depende de qué has escuchado en los últimos minutos/horas.

SECCIÓN 5 · ERRORES Y MALENTENDIDOS COMUNES

Error 1: Usar LSTM para problemas de NLP donde hay modelos preentrenados de Transformer disponibles. Para clasificación de texto, extracción de entidades, análisis de sentimiento o cualquier tarea de NLP estándar, un BERT o RoBERTa preentrenado con fine-tuning superará a cualquier LSTM en la mayoría de los datasets con menos tiempo de desarrollo.

Error 2: No normalizar las series temporales antes de entrenar una LSTM. Las LSTM son sensibles a la escala de los datos. Series temporales con escalas muy diferentes (temperatura en grados vs presión en bares vs corriente en amperios) deben normalizarse independientemente antes de alimentarlas a la red.

Error 3: Usar el error en el último paso temporal como única métrica. En predicción de series temporales multistep (predecir los próximos 10 pasos), el error en el paso 1 es siempre menor que en el paso 10. Evaluar solo el error promedio puede ocultar que el modelo es muy bueno a corto plazo pero muy malo a largo plazo.

SECCIÓN 6 · APLICACIÓN PRÁCTICA

Para construir un modelo LSTM de predicción de series temporales en PyTorch: define el modelo (nn.LSTM + nn.Linear), prepara los datos como ventanas deslizantes (ventana de n pasos pasados → 1 o más pasos futuros), normaliza cada variable independientemente con StandardScaler, entrena con teacher forcing para el primer entrenamiento y scheduled sampling para refinamiento, y evalúa con MAE y RMSE en el conjunto de test respetando el orden temporal (nunca usar k-fold estándar en series temporales).

Para series temporales empresariales, considera primero modelos estadísticos simples como ARIMA o Exponential Smoothing como baseline: son interpretables, rápidos y frecuentemente suficientemente buenos para horizontes cortos. Escala a LSTM/GRU cuando necesites capturar dependencias no lineales complejas o cuando tengas muchas series correlacionadas. Prophet de Meta es una excelente alternativa cuando las series tienen estacionalidad clara y eventos especiales conocidos.

SECCIÓN 7 · RELACIÓN CON OTROS MÓDULOS

- M1-T8 (Transformer): los Transformers reemplazaron a las RNNs en NLP. Entender las RNNs hace más claro por qué la atención fue un avance tan importante.
- M5 (LLMs): los primeros modelos de lenguaje (antes de 2017) eran LSTMs. Los LLMs modernos son Transformers, pero el concepto de estado oculto como memoria se preserva en los State Space Models.
- M8 (LLMOps): la monitorización de series temporales de métricas de modelos en producción puede usar LSTMs para detectar patrones anómalos de drift.

Las RNNs procesan secuencias con un estado oculto que actúa como memoria. Las LSTMs resuelven el problema del gradiente desvaneciente con compuertas que controlan qué retener y qué olvidar, capturando dependencias de cientos de pasos. Las GRUs son LSTMs simplificadas con rendimiento similar y menor coste computacional. Las BiRNNs procesan en ambas direcciones para mejor contexto. Los Transformers han reemplazado a las RNNs en NLP, pero las LSTMs siguen siendo la solución correcta para series temporales industriales, procesamiento online y recursos limitados. Los State Space Models (Mamba) son el sucesor emergente con complejidad $O(n)$.